

Multi-Threading for the Parking System

Software Design & Programming

Object-Oriented Programming II

Joseph DiBiasi

University of Denver College of Professional Studies

5/30/2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Bobbie Kite, MLS

Introduction

Our initial attempt at demonstrating data serialization was a success. We were able to create a workflow that allowed a user, connecting via our server client, to issue commands to the backend server. This allowed the registration of both customers and cars, proving our concept of serialization, but the potential customer base for utilizing the university parking system is much larger than our current system could handle. This development cycle we are focused on expanding the initial communication structure so that the server can handle multiple connections at once.

Requirements

To demonstrate that our server can handle more than one connection, we need to implement a way for our current structure to handle more than one connection thread at a time. Multi-threading brings its own challenges, while our current design might meet most needs during periods of low traffic, we need to ensure our server can handle high stress loads and handle them correctly. Therefore, we need to rework some of the existing code structure so that concurrent threads will still be operating safely despite working with the same variables. Finally, we need a way to validate that our new structure is indeed capable of handling multiple threads by creating tests capable of mocking multiple users simultaneously using the registration commands.

High-Level Design

The first step in optimizing our current design for multi-threading was in breaking up our initial server class, removing the client handler functionality and placing it in its own class. This

rework provided immediate benefits regarding the separation of concerns for the two classes. The server class was handling a lot of work; while this is fine for a demonstration, the class could become overwhelmingly complicated to maintain as our application grows. This new design allows the server class to focus more exclusively on its server duties: starting it up, accepting new connections, and closing those connections when it shuts down. The actual lifecycle of a single thread is no longer its concern as the server relies on the new client handler class to process the request for each thread, handle the response, then close the socket. The server now uses a method to handle the input and output of client streams; this separation of concerns is more to prepare for future development. If we later decide to authenticate user permissions or validate the incoming data, we could handle that within this method while leaving the client handler class logic alone.

Implementation

The main work implementing multi-threading revolved around the changes to the server class. While this class initially oversaw a single thread from start to end, we changed it to instead manage a thread pool to handle multiple connections simultaneously. We wanted this connection pool to be dynamic to an extent but also bounded in reality by the machines we are using to operate our testing. By using the available runtime processors, we can achieve a nice middle ground. Once we are ready to deploy this to a hosted environment, we can use a connection pool that we know will be supported by our environment. The server also utilizes a client handler method to manage the input and output streams from the client handler class. This keeps the class separate from business logic, making later modifications easier.

The client handler class itself is an extension of runnable. The central method in this class is the override of the run command. You may notice that the socket is not manually closed, this is intentional as we use a try-catch with resources. Passing the socket into the initial try statement means that upon completion of the try-catch the resource will be automatically closed. This saves us a little extra overhead by reducing the need for a finally clause at the end of the statement.

Our parking service and parking office classes generally were able to maintain their same structure despite switching to a multi-threading server-client process. Instead of changing their structure or logic, we simply put in a few safety guards to handle concurrent modifications. The parking service class had its command list replaced with a concurrent hash map. This allows safe read-write actions by multiple threads at once. The parking office had the synchronized keyword added to all methods pertaining to the modification of the customer and car lists. Since both the customer and car lists are mutable variables being modified during a session, we needed to ensure that each thread would still handle changes to them in an orderly fashion. Synchronized handles the locked and unlocking of these variables so that only a single thread is acting on them at a time. Testing and verification of the application was done using test cases. A special server test class was created to validate concurrent thread attempts; this test started a server and attempts to act with 10 threads at once.

```

::: Command ::: CAR CAR, ownerid=7a76594f-0a59-4ddc-8343-1e0c84c2e579, license=LIC9, cartype=SUV
May 31, 2026 12:59:44 PM clients.ServerClient runCommand
INFO: You are now connected to: 127.0.0.1
Car registered successfully:
Owner ID: 7a76594f-0a59-4ddc-8343-1e0c84c2e579
License: LIC9
Car Type: SUV
(handled in 85 ms)
Average time per successful client: 538 ms
Total elapsed time: 8558.84 ms
Summary: 10/10 succeeded, 0 failed.
All clients finished.

```

Figure 1. UI Test Script Validation of 10 Customer & Car Registrations

Summary

Making the leap from a single connection to multiple is an adjustment, but it is far easier to prepare for this change than be forced to adapt to it by necessity. We know that our eventual customer base will be much larger than the initial test cases we have operated with to validate our work. An initial attempt at addressing this was done via script. We created a script that can accept variable connections and return results, this lets us prepare for a larger number of users, such as we could expect on release. However, we started to run into bottlenecks at higher numbers that looked to be a result of our script instead of our code as our connections were resolved quickly but the results were taking significantly longer to be collated by the script. We know we have more work to do in this area before we are ready to release into our larger planned customer base; being aware of the challenges of multi-threading means we can take them head on from the start. Multi-threading may be complicated, but we are ready for the challenge!

```
::: Command ::: CAR CAR, ownerId=eff8b722-d91f-4ecc-88b2-1cbbb4af54d4, license=LIC4, cartype=SUV
May 31, 2026 12:59:27 PM clients.ServerClient runCommand
INFO: You are now connected to: 127.0.0.1
Car registered successfully:
Owner ID: eff8b722-d91f-4ecc-88b2-1cbbb4af54d4
License: LIC4
Car Type: SUV
(handled in 60 ms)
Average time per successful client: 531.8 ms
Total elapsed time: 1069.75 ms
Summary: 5/5 succeeded, 0 failed.
All clients finished.
```

Figure 2. UI Test Script Validation of only 5 Users sees 8x Performance Increase